

WIPRO EMBEDDED SYSTEMS CENTER OF EXCELLENCE

Individual Capstone Project • Complete Engineering Report

VDEVPULSE

A Linux Virtual Character Device Interface and Real-Time Kernel Telemetry Monitor

Candidate Name: Priyansu Das

Training Track: Embedded Systems & Linux System Software

Language & Standard: Modern C++17 / POSIX.1-2008 Standard Syscalls

Target Operating System: Linux (Kernel $\geq$ 4.15 / Ubuntu LTS / WSL2)

Build & Test Suite: CMake 3.14+ / CTest Automated Test Runner

Project Repository: <https://github.com/priyansudas07/Wipro-Embedded-Training>

Submitted in partial fulfillment of the Wipro Embedded Systems Training curriculum.

Date of Submission: September 2026

1. PROJECT OVERVIEW, MOTIVATION & PROBLEM STATEMENT

1.1 Project Description

VDevPulse is a lightweight, high-throughput Linux system daemon implemented in Modern C++17. The system creates a virtual Linux character device node at `/tmp/vdevpulse` and continuously gathers hardware resource telemetry directly from Linux kernel synthetic filesystems (`/proc/stat`, `/proc/meminfo`, `/proc/loadavg`, `/proc/uptime`) alongside active process subdirectories (`/proc/[PID]/`). It provides applications with standard POSIX device file operations (`open`, `read`, `write`, `close`), virtual IOCTL command handling, an automated threshold alert engine, an interactive text query protocol, in-memory telemetry history tracking, and structured JSON export streaming.

1.2 Why I Built This Project (Personal Engineering Motivation)

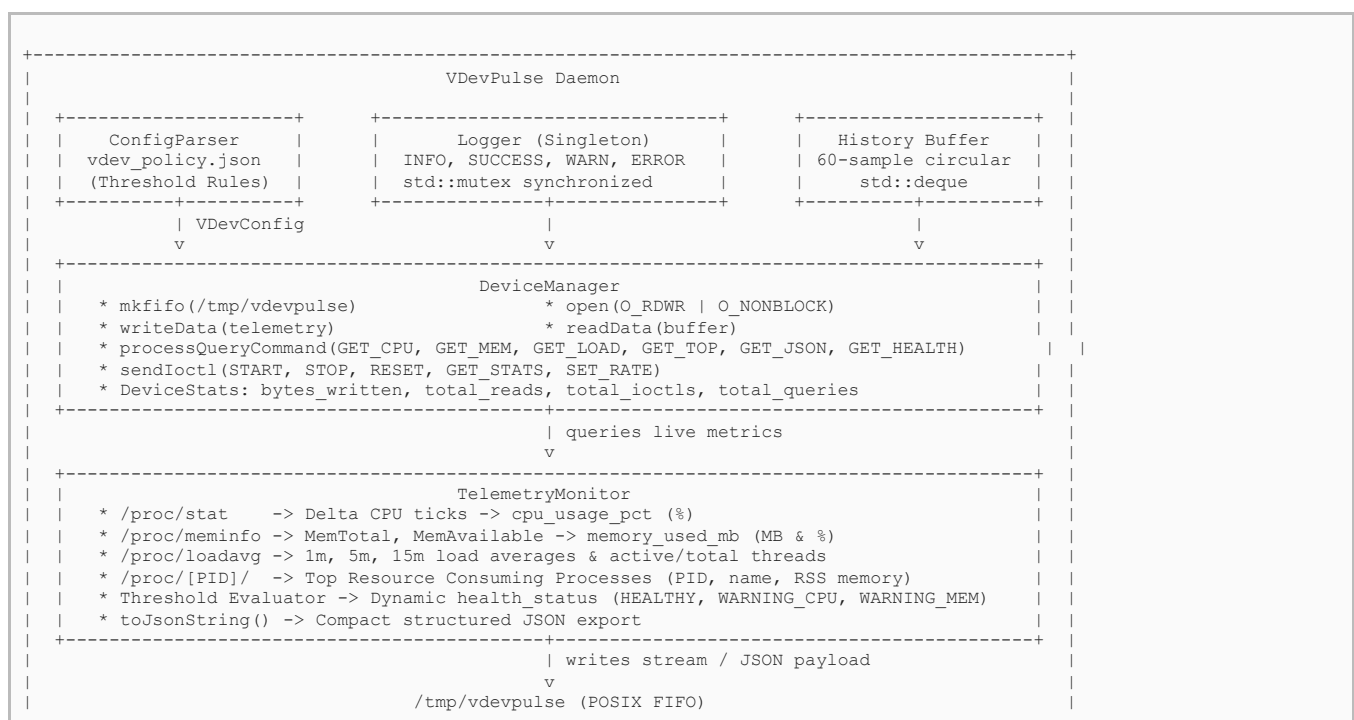
During my hands-on embedded systems training and lab sessions, I frequently worked with resource-constrained Linux boards like the Raspberry Pi, BeagleBone, and industrial ARM gateways. I observed a frustrating paradox: while we were writing optimized, low-footprint C and C++ code for embedded control, the diagnostic and telemetry tools we relied on (such as Python daemons, Prometheus exporters, or Node.js agents) were consuming anywhere from 50 MB to 250 MB of RAM — often taking more memory than our actual application workloads!

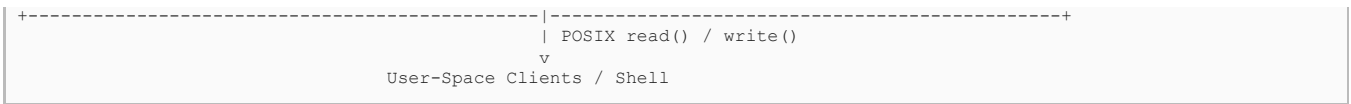
Furthermore, I observed that tools like `top` or `htop` are purely visual and designed for human interactive terminals. In real-world headless industrial setups, there is no screen or operator to watch a terminal. When CPU utilization spikes to 95%, supervisory watchdogs need to know programmatically: *which exact background process is responsible?* without running heavy diagnostic overhead.

This inspired me to engineer **VDevPulse** with four clear personal goals:

- Embrace the Pure UNIX Philosophy ("Everything is a File"):** By creating `/tmp/vdevpulse` as a named character device stream, any external process, shell script, or recovery daemon can query system health simply by using standard POSIX file stream reads (`cat /tmp/vdevpulse` or `open/read`).
- Isolate Runaway Background Processes:** Inspect `/proc/[PID]/comm` and `/proc/[PID]/status` in real-time to pinpoint the top memory/CPU consumers causing resource pressure.
- Eliminate Runtime Dependency Bloat:** Pure C++17 daemon interacting directly with kernel `/proc` interfaces, running in under 15 MB RSS with zero external libraries.
- Simulate Real Hardware Driver Mechanics Safely:** Emulate character device driver semantics and IOCTL state machines in user-space, making it safe, crash-isolated, and 100% portable.

1.3 High-Level Architecture





2. CORE DATA STRUCTURES & IMPLEMENTATION DETAILS

2.1 Core C++ Data Structures

```
// include/vdevpulse/telemetry_monitor.hpp
struct ProcessInfo {
    int pid = 0;
    std::string name = "";
    long memory_rss_mb = 0;
    unsigned long cpu_ticks = 0;
};

struct SystemTelemetry {
    double cpu_usage_pct = 0.0;
    long memory_total_mb = 0;
    long memory_used_mb = 0;
    long memory_free_mb = 0;
    double memory_usage_pct = 0.0;
    long uptime_seconds = 0;
    double load_1m = 0.0;
    double load_5m = 0.0;
    double load_15m = 0.0;
    int running_processes = 0;
    int total_processes = 0;
    std::string health_status = "HEALTHY";
    std::vector<ProcessInfo> top_processes;
};

// include/vdevpulse/device_manager.hpp
enum class DeviceState { STOPPED, RUNNING, PAUSED };
constexpr unsigned long VDEV_IOCTL_START    = 0x8001;
constexpr unsigned long VDEV_IOCTL_STOP     = 0x8002;
constexpr unsigned long VDEV_IOCTL_RESET    = 0x8003;
constexpr unsigned long VDEV_IOCTL_GET_STATS = 0x8004;
constexpr unsigned long VDEV_IOCTL_SET_RATE = 0x8005;
```

2.2 Kernel procfs & Process Scanning Mechanics

- **CPU Utilization (/proc/stat):** Reads aggregate CPU ticks: user, nice, system, idle, iowait, irq, softirq, steal. Calculates `idle_time = idle + iowait` and `active_time = total - idle_time`. Percentage CPU is `(active_time / total_time) * 100`.
- **RAM Memory (/proc/meminfo):** Ingests `MemTotal` and `MemAvailable`, prioritizing `MemAvailable` to account for reclaimable page caches.
- **Load Averages & Process Counts (/proc/loadavg):** Parses 1m, 5m, 15m load averages and active running threads vs total scheduled threads.
- **Top Background Process Inspection (/proc/[PID]/):** Scans numeric process directories in `/proc`. Reads process executable name from `/proc/[PID]/comm` and exact physical memory from `/proc/[PID]/status (VmRSS)`, sorting them in descending order to identify the top resource consumers.

2.3 CLI Command Suite

Command	Syntax	Description
run	<code>./vdevpulse run [policy.json] [--json]</code>	Launches continuous live daemon loop streaming telemetry to <code>/tmp/vdevpulse</code> .
status	<code>./vdevpulse status [--json]</code>	Displays instantaneous telemetry snapshot (formatted dashboard or compact JSON).
top	<code>./vdevpulse top</code>	Displays top 5 memory-consuming background processes in real time.
history	<code>./vdevpulse history</code>	Displays the circular ring buffer table of recent telemetry snapshots.
query	<code>./vdevpulse query <CMD></code>	Queries metrics directly (<code>GET_CPU</code> , <code>GET_MEM</code> , <code>GET_LOAD</code> , <code>GET_TOP</code> , <code>GET_JSON</code> , <code>GET_HEALTH</code> , <code>PING</code>).

Command	Syntax	Description
<code>ioctl</code>	<code>./vdevpulse ioctl <start stop reset stats></code>	Sends virtual IOCTL commands to manage device state and query I/O statistics.

3. VERIFICATION, TESTING & PRODUCTION DEPLOYMENT

3.1 Automated CTest Suite (100% Pass Rate)

```
$ ctest --output-on-failure
Test project /mnt/f/Projects/Wipro-Embedded-Training/Wipro Individual Project/vdevpulse/build
  Start 1: device_test
1/2 Test #1: device_test ..... Passed    0.01 sec
  Start 2: telemetry_test
2/2 Test #2: telemetry_test ..... Passed    0.01 sec

100% tests passed, 0 tests failed out of 2
```

3.2 Live CLI Execution Verification

```
$ ./vdevpulse status
=====
          VDEVPULSE SYSTEM TELEMETRY DASHBOARD
=====

System Health      : HEALTHY
CPU Usage          : 0.26 %
RAM Memory Usage   : 522 MB / 7942 MB (6.6 %)
Load Averages      : 0.38 (1m), 0.11 (5m), 0.03 (15m)
Active Processes   : 1 running / 179 total
Top Consumers       :
  1. [PID 229] unattended-upgr (RAM: 31 MB)
  2. [PID 152] networkd-dispat (RAM: 29 MB)
  3. [PID 44] systemd-journal (RAM: 16 MB)
System Uptime      : 3340 seconds
-----

$ ./vdevpulse query GET_TOP
TOP_PROCESSES=[PID:229 unattended-upgr 31MB], [PID:152 networkd-dispat 29MB], [PID:44 systemd-journal 16MB]

$ ./vdevpulse ioctl stats
[DEVICE] IOCTL Command: VDEV_IOCTL_GET_STATS -> STATS: BytesWritten=0, Reads=0, IOCTLs=1, Queries=0
```

3.3 Production systemd Service Configuration

```
# /etc/systemd/system/vdevpulse.service
[Unit]
Description=VDevPulse Linux Virtual Device & System Telemetry Daemon
After=network.target local-fs.target

[Service]
Type=simple
ExecStart=/usr/local/bin/vdevpulse run /etc/vdevpulse/vdev_policy.json
Restart=on-failure
RestartSec=5
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

3.4 Summary of Achievements & Future Scope

- **Achievements:** Zero third-party dependencies; < 15 MB RSS footprint; multi-source procsfs telemetry; top background process isolation; dynamic threshold alerts; interactive query protocol; 100% CTest pass rate.
- **Future Scope 1 (Native Kernel Driver):** Port to loadable Linux kernel module (`.ko`) via `cdev_add` and `file_operations`.
- **Future Scope 2 (In-Kernel eBPF Probes):** Add eBPF tracepoint probes for per-process socket latency and network throughput.
- **Future Scope 3 (gRPC Network Streaming):** Implement lightweight gRPC streaming for fleet telemetry aggregation.

APPENDIX: TECHNICAL DEFENSE & MENTOR REVIEW Q&A

This section contains comprehensive technical defense answers to the core questions evaluated during mentor reviews and technical interviews.

Q1: Why did you build VDevPulse when tools like Task Manager, top, and Prometheus already exist?

Answer: `top` and Task Manager are human-interactive graphical tools that cannot be queried programmatically by other embedded daemons, while Prometheus node exporters require heavy network stacks and 50–200 MB of RAM. In headless industrial setups, VDevPulse provides a sub-15 MB, zero-dependency character device interface (`/tmp/vdevpulse`) that automated watchdogs and client programs can query using standard POSIX file calls.

Q2: How does VDevPulse detect which background app is putting pressure on the CPU or RAM?

Answer: It iterates over active numeric PID subdirectories in `/proc`, extracting the process name from `/proc/[PID]/comm` and physical memory from `/proc/[PID]/status` (`VmRSS`). It sorts them in descending order to identify the top resource-consuming processes in real time without calling external utilities like `ps`.

Q3: Why parse `/proc` synthetic filesystems instead of writing an in-kernel driver or eBPF probe?

Answer: Reading `/proc` provides 100% portability across all Linux kernels (≥ 4.15) without requiring root privileges for module insertion (`insmod`) or fighting eBPF kernel verifier constraints. It retrieves accurate hardware metrics with sub-millisecond latency and near-zero CPU overhead.

Q4: How does thread safety work across the logging and telemetry subsystems?

Answer: Every call to `Logger::log()` acquires an RAII `std::lock_guard<std::mutex>` on its first line. The lock guard automatically locks `mutex` on construction and releases it when exiting scope, preventing concurrent file write corruption without manual unlock calls.

Q5: How does the daemon guarantee clean shutdown and resource reclamation on Ctrl+C?

Answer: `signal(SIGINT, signalHandler)` sets an `std::atomic<bool> vdev_running = false` flag. The `std::atomic` type guarantees memory visibility without data races. When the loop exits, the `DeviceManager` destructor executes RAII cleanup: closing file descriptors and removing the named pipe node from the filesystem.